

JavaScript Intermediate — Object Creation, Constructor Functions & new

Section 15.2

আগের section-এ আমরা শিখেছি:

```
Object
  ↓
[[Prototype]]
  ↓
Prototype Chain
  ↓
Object.prototype
```

এখন প্রশ্ন:

এই **prototype relationship** তৈরি করব কীভাবে?

এখানে আসবে:

- `Object.create()`
- Constructor Functions
- `this`
- Constructor-এর `prototype`
- `new`
- `new.target`
- Constructor return behavior
- Factory Function vs Constructor Function

1. Object তৈরি করার বিভিন্ন উপায়

JavaScript-এ object তৈরি করার অনেক উপায় আছে:

```
const user = {
  name: "Shariar"
};
```

অথবা:

```
const user = new Object();
```

অথবা:

```
const user = Object.create(proto);
```

অথবা Constructor Function:

```
function User(name) {  
  this.name = name;  
}  
  
const user = new User("Shariar");
```

অথবা modern class:

```
class User {  
  constructor(name) {  
    this.name = name;  
  }  
}
```

Class নিয়ে বিস্তারিত পরের document-এ আসবে।

2. Object.create(proto)

`Object.create()` এমন একটি object তৈরি করে যার prototype তুমি explicitly নির্ধারণ করতে পারো।

Syntax:

```
Object.create(proto);
```

Example:

```
const animal = {  
  eats: true  
};
```

```
const dog = Object.create(animal);  
  
console.log(dog.eats);
```

Output:

```
true
```

কারণ:

```
dog  
↓  
animal  
↓  
Object.prototype  
↓  
null
```

3. `Object.create()` আসলে কী করে?

ধরো:

```
const parent = {  
  role: "user"  
};  
  
const child = Object.create(parent);
```

Conceptually:

```
child  
├─ own properties  
└─ [[Prototype]]  
    ↓  
  parent
```

অর্থাৎ `child` -এর prototype হলো `parent` ।

Check:

```
console.log(
  Object.getPrototypeOf(child) === parent
);
```

Output:

```
true
```

4. `Object.create()` দিয়ে Inheritance

```
const animal = {
  eat() {
    console.log("Eating");
  }
};

const dog = Object.create(animal);

dog.bark = function () {
  console.log("Barking");
};

dog.eat();
dog.bark();
```

Output:

```
Eating
Barking
```

এখানে:

```
dog
├─ bark()
└─ prototype
    ↓
```

```
animal
└─ eat()
```

5. Own Property তৈরি করা

`Object.create()` prototype set করে, কিন্তু child object-এর নিজের property তুমি পরে add করতে পারো।

```
const animal = {
  eats: true
};

const dog = Object.create(animal);

dog.name = "Buddy";
dog.barks = true;
```

এখন:

```
dog
├─ name
├─ barks
└─ [[Prototype]]
    ↓
  animal
    └─ eats
```

6. `Object.create(null)`

`Object.create()` -এ `null` দিলে prototype-less object পাওয়া যায়।

```
const dictionary = Object.create(null);
```

তার prototype:

```
console.log(Object.getPrototypeOf(dictionary));
```

Output:

```
null
```

Chain:

```
dictionary
  ↓
  null
```

এখানে `Object.prototype` নেই।

তাই:

```
dictionary.toString;
```

হবে:

```
undefined
```

এটা dictionary-like data structure-এর কিছু use case-এ useful।

7. `Object.create()` বনাম **Object Literal**

Object Literal

```
const user = {
  name: "Sharlar"
};
```

এখানে default prototype সাধারণত:

```
Object.prototype
```

```
Object.create()
```

```
const user = Object.create(customPrototype);
```

এখানে তুমি prototype explicitly define করছ।

8. Constructor Function কী?

Class আসার আগে JavaScript-এ object তৈরির জন্য constructor functions খুব common ছিল।

Example:

```
function User(name, age) {  
  this.name = name;  
  this.age = age;  
}
```

এটা একটি **constructor function** হিসেবে ব্যবহার করা যায়।

তারপর:

```
const user1 = new User("Shariar", 31);  
const user2 = new User("Rahim", 25);
```

দুইটা আলাদা object তৈরি হবে।

9. Constructor Function Naming Convention

Constructor function-এর নাম সাধারণত PascalCase-এ লেখা হয়:

```
function User() {}  
function BankAccount() {}  
function Product() {}
```

এটা JavaScript-এর hard rule নয়।

এটা convention।

এতে বোঝা যায়:

এই function-টা সাধারণ function call করার জন্য নয়; `new` দিয়ে instance তৈরি করার জন্য intended ।

10. Constructor Function-এর `this`

Constructor function:

```
function User(name, age) {  
  this.name = name;  
  this.age = age;  
}
```

তারপর:

```
const user = new User("Shariar", 31);
```

এখানে `this` নতুন তৈরি হওয়া object-কে refer করে।

Conceptually:

```
new User(...)  
  ↓  
new object  
  ↓  
this → new object
```

তারপর:

```
this.name = "Shariar";  
this.age = 31;
```

object-এর properties তৈরি করে।

11. Constructor Function Example

```
function User(name, age) {  
  this.name = name;  
  this.age = age;  
  
  this.greet = function () {  
    console.log(`Hello ${this.name}`);  
  };  
}  
  
const user = new User("Shariar", 31);  
  
console.log(user.name);  
console.log(user.age);  
  
user.greet();
```

Output:

```
Shariar  
31  
Hello Shariar
```

12. Constructor-এর Prototype

প্রতিটি normal function-এর একটি `prototype` property থাকে।

```
function User(name) {  
  this.name = name;  
}  
  
console.log(User.prototype);
```

এটা একটি object।

Constructor function-এর prototype-এ methods রাখা যায়।

13. Prototype-এ Method রাখা

Better approach:

```
function User(name) {  
  this.name = name;  
}  
  
User.prototype.greet = function () {  
  console.log(`Hello ${this.name}`);  
};
```

তারপর:

```
const user1 = new User("Shariar");  
const user2 = new User("Rahim");  
  
user1.greet();  
user2.greet();
```

দুই instance-ই একই prototype method ব্যবহার করবে।

14. কেন Prototype-এ Method রাখব?

এই approach:

```
function User(name) {  
  this.name = name;  
  
  this.greet = function () {  
    console.log(`Hello ${this.name}`);  
  };  
}
```

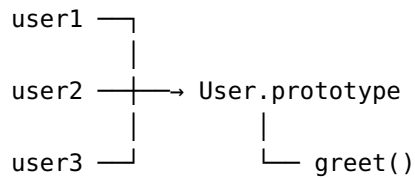
এখানে প্রতিবার `new User()` করলে নতুন `greet` function তৈরি হতে পারে।

কিন্তু:

```
User.prototype.greet = function () {
  console.log(`Hello ${this.name}`);
};
```

এখানে method shared ।

Mental model:



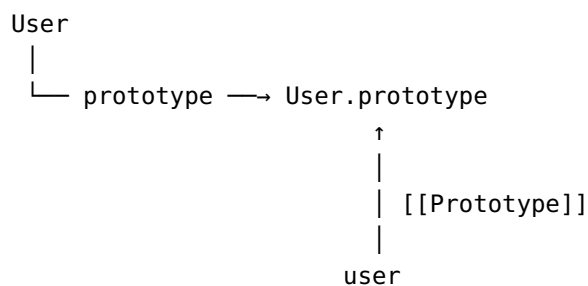
15. Constructor + Prototype Relationship

এটা খুব গুরুত্বপূর্ণ:

```
function User(name) {
  this.name = name;
}

const user = new User("Shariar");
```

Relationship:



অর্থাৎ:

```
Object.getPrototypeOf(user) === User.prototype
```

হবে:

```
true
```

16. `user.__proto__` vs `User.prototype`

```
console.log(user.__proto__ === User.prototype);
```

Output:

```
true
```

এখানে:

```
user.__proto__
```

বলছে:

user-এর prototype কে?

আর:

```
User.prototype
```

বলছে:

User constructor-এর prototype object কোনটা?

`new User()` করার সময় এই দুইটার মধ্যে relationship তৈরি হয়।

17. The `new` Keyword

`new` শুধু "নতুন object বানায়" বললে পুরো বিষয়টা বোঝা হয় না।

ধরো:

```
function User(name) {  
  this.name = name;  
}  
  
const user = new User("Shariar");
```

`new` conceptually কয়েকটি গুরুত্বপূর্ণ কাজ করে।

18. `new` — Step 1: New Empty Object

প্রথমে একটি নতুন object তৈরি হয়।

Conceptually:

```
{}
```

19. `new` — Step 2: Prototype Connect

নতুন object-এর `[[Prototype]]` constructor-এর `.prototype` -এর সাথে connect হয়।

```
new object  
  |  
  ↓  
User.prototype
```

অর্থাৎ:

```
Object.getPrototypeOf(user) === User.prototype
```

20. `new` — Step 3: Constructor Call with `this`

Constructor function execute হয় এবং `this` নতুন object-কে refer করে।

```
function User(name) {  
  this.name = name;  
}
```

Conceptually:

```
this → new object
```

তাই:

```
this.name = name;
```

নতুন object-এর মধ্যে:

```
name
```

তৈরি করে।

21. new — Step 4: Return Object

শেষে নতুন object return হয়।

তাই:

```
const user = new User("Shariar");
```

এর result হলো newly created object।

22. new Mental Model

সহজভাবে:

```
new User("Shariar")  
  ↓  
1. Create empty object  
  ↓
```

2. Connect prototype
↓
3. Call User with this = new object
↓
4. Return object

এটাই **new** বোঝার সবচেয়ে important mental model।

23. Conceptual Implementation of **new**

আমরা পুরো engine implementation লিখছি না, কিন্তু conceptually:

```
function createInstance(Constructor, ...args) {  
  const instance = Object.create(Constructor.prototype);  
  
  const result = Constructor.apply(instance, args);  
  
  if (result !== null && typeof result === "object") {  
    return result;  
  }  
  
  return instance;  
}
```

এটা exact ECMAScript implementation নয়।

শুধু **new**-এর core idea বোঝানোর জন্য।

24. `Object.create(Constructor.prototype)`

এই line:

```
const instance = Object.create(Constructor.prototype);
```

খুব important।

কারণ এটিই prototype relationship তৈরি করার concept দেখায়।

তারপর:

```
Constructor.apply(instance, args);
```

constructor-এর ভিতরে:

```
this
```

কে `instance` হিসেবে ব্যবহার করার ধারণা বোঝায়।

25. Constructor Return না করলে

```
function User(name) {  
  this.name = name;  
}  
  
const user = new User("Sharar");  
  
console.log(user.name);
```

Output:

```
Sharar
```

Constructor explicitly কিছু return করেনি।

তাই নতুন object-টাই result হয়।

26. Constructor Primitive Return করলে

```
function User(name) {  
  this.name = name;  
  
  return 100;  
}  
  
const user = new User("Sharar");  
  
console.log(user.name);
```

Output:

Shariar

Primitive return:

```
number
string
boolean
null
undefined
symbol
bigint
```

সাধারণত newly created instance replace করে না।

27. Constructor Object Return করলে

এখানে behavior interesting।

```
function User(name) {
  this.name = name;

  return {
    name: "Other"
  };
}

const user = new User("Shariar");

console.log(user.name);
```

Output:

Other

কারণ constructor explicitly একটি object return করেছে।

`new` -এর result সেই returned object হয়ে যায়।

28. Constructor Return Rule

সহজ mental model:

```
Constructor return primitive
      ↓
Ignore it
      ↓
Use new instance
```

কিন্তু:

```
Constructor return object
      ↓
Use returned object
```

এটা constructor functions-এর একটি important behavior ।

29. Constructor Function vs Normal Function

Constructor:

```
function User(name) {
  this.name = name;
}

const user = new User("Shariar");
```

Normal function:

```
function greet(name) {
  console.log(name);
}

greet("Shariar");
```

Difference mainly usage/context-এর।

একই JavaScript function technically constructor হিসেবে ব্যবহার করা যায় যদি `new` দিয়ে call করা যায় এবং function constructor-compatible হয়।

30. `new` ছাড়া Constructor Call

```
function User(name) {  
  this.name = name;  
}  
  
const user = User("Sharir");
```

এখানে `new` নেই।

তাই এটা constructor call হিসেবে কাজ করছে না।

Strict mode/module environment-এ `this` undefined হতে পারে।

ফলে:

```
this.name = name;
```

error দিতে পারে।

তাই constructor function-এর intended usage:

```
new User(...)
```

31. Constructor Function-এর Prototype Method

Complete example:

```
function User(name, age) {  
  this.name = name;  
  this.age = age;  
}  
  
User.prototype.greet = function () {  
  console.log(`Hello, I am ${this.name}`);  
}
```

```

};

User.prototype.isAdult = function () {
  return this.age >= 18;
};

const user1 = new User("Shariar", 31);
const user2 = new User("Rahim", 25);

user1.greet();
user2.greet();

console.log(user1.isAdult());

```

Prototype chain:

```

user1
  ↓
User.prototype
  ├── greet()
  └── isAdult()
  ↓
Object.prototype
  ↓
null

```

32. Instance Property vs Prototype Property

এই distinction খুব important।

```

function User(name) {
  this.name = name;
}

User.prototype.greet = function () {};

```

এখানে:

```

name → instance property
greet → prototype property

```

Check:

```
console.log(Object.hasOwn(user, "name"));
```

Result:

```
true
```

কিন্তু:

```
console.log(Object.hasOwn(user, "greet"));
```

Result:

```
false
```

কারণ `greet` inherited।

33. `in` vs `Object.hasOwn()`

```
console.log("name" in user);  
console.log("greet" in user);
```

দুটোই:

```
true
```

কারণ `in` prototype chain-এও search করে।

কিন্তু:

```
Object.hasOwn(user, "greet");
```

হবে:

false

34. new.target

`new.target` constructor/function-এর ভিতরে বোঝাতে পারে function-টি `new` দিয়ে call হয়েছে কি না।

Example:

```
function User() {  
  console.log(new.target);  
}  
  
new User();
```

এখানে `new.target` হবে `User`।

কিন্তু:

```
User();
```

এর ক্ষেত্রে `new.target` হবে:

undefined

35. new.target কেন Useful?

Constructor function-কে accidentally normal function হিসেবে call করা হচ্ছে কিনা detect করতে ব্যবহার করা যায়।

```
function User(name) {  
  if (!new.target) {  
    throw new Error("User must be called with new");  
  }  
  
  this.name = name;  
}
```

এখন:

```
new User("Shariar");
```

valid।

কিন্তু:

```
User("Shariar");
```

error করবে।

36. Factory Function কী?

Factory function হলো এমন normal function যেটা object তৈরি করে return করে।

```
function createUser(name, age) {  
  return {  
    name,  
    age  
  };  
}  
  
const user = createUser("Shariar", 31);
```

এখানে `new` দরকার নেই।

37. Factory vs Constructor

Factory

```
function createUser(name) {  
  return {  
    name  
  };  
}  
  
const user = createUser("Shariar");
```

Constructor

```
function User(name) {  
  this.name = name;  
}  
  
const user = new User("Shariar");
```

38. Factory Function vs Constructor Function

Feature	Factory	Constructor
<code>new</code> দরকার	✗	✓
<code>this</code> ব্যবহার	সাধারণত না	হ্যাঁ
Explicit object return	সাধারণত হ্যাঁ	সাধারণত না
Prototype sharing	manually design করতে হয়	built-in constructor prototype pattern
Syntax	সাধারণ function	PascalCase convention
Modern code	অনেক ক্ষেত্রে সহজ	Class-এর আগের common pattern

39. `Object.create()` vs Constructor

`Object.create()`

```
const animal = {  
  eats: true  
};  
  
const dog = Object.create(animal);
```

এখানে prototype relationship explicitly define করা হচ্ছে।

Constructor

```
function Dog(name) {  
  this.name = name;  
}
```

```
const dog = new Dog("Buddy");
```

এখানে:

```
dog
↓
Dog.prototype
```

automatically তৈরি হয় `new` mechanism-এর মাধ্যমে।

40. Constructor-এর Prototype Object-এর Constructor

Defaultভাবে:

```
function User() {}

console.log(User.prototype.constructor === User);
```

Output:

```
true
```

অর্থাৎ:

```
User
↓
User.prototype
↓
constructor → User
```

এই `constructor` property prototype object-এর একটি property।

41. Prototype Method যোগ করলে

```
function User(name) {
  this.name = name;
}
```

```
}

User.prototype.greet = function () {
  console.log("Hello");
};
```

constructor সাধারণত এখনও:

```
User.prototype.constructor === User
```

থাকবে।

42. Prototype পুরো Replace করার সময় সাবধান

এই pattern:

```
User.prototype = {
  greet() {
    console.log("Hello");
  }
};
```

করলে default **constructor** property হারিয়ে যেতে পারে।

তখন:

```
User.prototype.constructor === User
```

আর **true** নাও হতে পারে।

তাই prototype পুরো replace করার সময় constructor relationship সম্পর্কে সচেতন থাকতে হবে।

43. Prototype Chain Full Example

```
function User(name) {
  this.name = name;
}
```

```
User.prototype.greet = function () {  
  console.log(`Hello ${this.name}`);  
};  
  
const user = new User("Shariar");
```

Chain:

```
user  
├── name = "Shariar"  
└── [[Prototype]]  
    ↓  
User.prototype  
├── greet()  
├── constructor → User  
└── [[Prototype]]  
    ↓  
Object.prototype  
├── toString()  
├── valueOf()  
├── hasOwnProperty()  
└── ...  
    ↓  
null
```

এটাই constructor + prototype system-এর core structure ।

44. Important Concept — Data vs Behavior

একটি useful design pattern:

```
Instance  
↓  
নিজের data  
↓  
Prototype
```

↓
shared behavior

Example:

```
function User(name) {  
  this.name = name;  
}  
  
User.prototype.greet = function () {  
  console.log(this.name);  
};
```

এখানে:

name → instance-specific data
greet() → shared behavior

এটা prototype-based object design-এর একটি fundamental idea ।

45. Common Mistakes

Mistake 1 — `prototype` হলো **object's prototype**

সবসময় না ।

`Constructor.prototype`

একটি property ।

কিন্তু:

`object.[[Prototype]]`

হলো object-এর actual prototype relationship ।

Mistake 2 — Constructor function মানেই class

না।

Constructor function হলো function-based object construction pattern।

Class হলো modern syntax যা prototype system-এর উপর কাজ করে।

Mistake 3 — `new` শুধু **object** তৈরি করে

Incomplete।

`new` conceptually:

```
create object
+
connect prototype
+
bind this
+
run constructor
+
return result
```

Mistake 4 — Prototype method প্রতিটি **object**-এর own property

না।

```
Object.hasOwn(user, "greet");
```

সাধারণত:

```
false
```

হবে।

Mistake 5 — Constructor-এ method রাখা সবসময় best

```
function User(name) {  
  this.name = name;  
  
  this.greet = function () {};  
}
```

এতে প্রতিটি instance-এর জন্য আলাদা function তৈরি হতে পারে।

Shared behavior হলে prototype-এ রাখা সাধারণত better।

46. Output Prediction

Question 1

```
const animal = {  
  eats: true  
};  
  
const dog = Object.create(animal);  
  
console.log(dog.eats);
```

Output কী?

Question 2

```
function User(name) {  
  this.name = name;  
}  
  
const user = new User("Shariar");  
  
console.log(user.name);  
console.log(  
  Object.getPrototypeOf(user) === User.prototype  
);
```

Output কী?

Question 3

```
function User() {}

User.prototype.greet = function () {};

const user = new User();

console.log(Object.hasOwn(user, "greet"));
console.log("greet" in user);
```

Output কী?

Question 4

```
function User() {
  this.name = "Shariar";

  return {
    name: "Rahim"
  };
}

const user = new User();

console.log(user.name);
```

Output কী এবং কেন?

Question 5

```
function User() {
  this.name = "Shariar";

  return 100;
}

const user = new User();
```

```
console.log(user.name);
```

Output কী এবং কেন?

47. Practice Tasks

Task 1 — `Object.create()`

একটি:

```
animal
```

object তৈরি করো যার:

```
eat()  
sleep()
```

থাকবে।

তারপর `Object.create(animal)` দিয়ে:

```
dog
```

তৈরি করো।

Dog-এর নিজের:

```
bark()
```

থাকবে।

Task 2 — Constructor Function

একটি:

User

constructor বানাও।

Properties:

name
age

তারপর:

greet()

method prototype-এ রাখো।

Task 3 — Prototype Sharing

দুইটি instance তৈরি করো:

```
const user1 = new User("Shariar", 31);  
const user2 = new User("Rahim", 25);
```

তারপর verify করো:

```
user1.greet === user2.greet
```

কেন `true` হচ্ছে explain করো।

Task 4 — Understand `new`

নিজের ভাষায় এই চারটি step explain করো:

1. Create object
2. Connect prototype
3. Bind this
4. Return object

48. Interview Questions

Basic

1. `Object.create()` কী?
2. Constructor function কী?
3. Constructor function এবং normal function-এর difference কী?
4. `new` keyword কী করে?
5. Constructor function-এর `this` কী refer করে?
6. Constructor-এর prototype কী?
7. Instance property এবং prototype property-এর difference কী?

Intermediate

1. `new` internally কীভাবে কাজ করে?
2. `Object.create(Constructor.prototype)` কেন গুরুত্বপূর্ণ?
3. `user.__proto__ === User.prototype` কেন true?
4. Constructor-এ method না রেখে prototype-এ method রাখার সুবিধা কী?
5. Factory function কী?
6. Factory function এবং constructor function-এর difference কী?
7. Constructor object return করলে কী হয়?
8. Constructor primitive return করলে কী হয়?
9. `new.target` কী?
10. Constructor function-কে `new` ছাড়া call করলে কী সমস্যা হতে পারে?
11. `User.prototype.constructor` কী?
12. Prototype পুরো replace করলে `constructor` property-এর কী হতে পারে?
13. `Object.create()` এবং `new` -এর মধ্যে difference কী?

49. Quick Revision

`Object.create()`

```
const child = Object.create(parent);
```

Meaning:

```
child.[[Prototype]] → parent
```

Constructor Function

```
function User(name) {  
  this.name = name;  
}
```

Constructor Prototype

```
User.prototype.greet = function () {};
```

Meaning:

```
instances  
  ↓  
User.prototype  
  ↓  
shared methods
```

new

```
const user = new User("Sharlar");
```

Mental model:

```
new  
  ↓  
Create object  
  ↓  
Connect prototype  
  ↓  
this = object  
  ↓  
Run constructor  
  ↓  
Return object
```

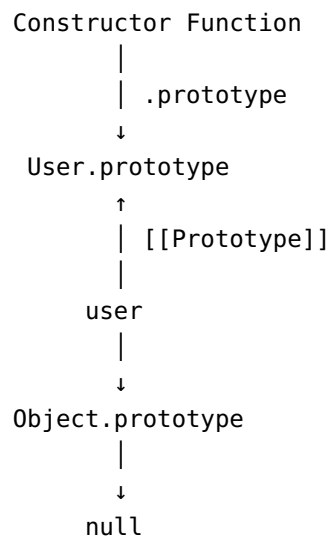
Factory

```
function createUser(name) {  
  return {  
    name  
  };  
}
```

No `new` required !

50. Final Mental Model

এই section-এর সবচেয়ে important relationship:



আর `new User()` করলে conceptually:

```
new User("Shariar")  
↓  
Create object  
↓  
[[Prototype]] = User.prototype  
↓  
User.call(newObject, "Shariar")  
↓  
this → newObject  
↓
```

```
newObject.name = "Shariar"  
↓  
return newObject
```

Mastery Checklist

- [] `Object.create()` বুঝি
- [] `Object.create(null)` বুঝি
- [] Constructor function বুঝি
- [] Constructor-এর `this` বুঝি
- [] Constructor-এর `.prototype` বুঝি
- [] Instance property vs prototype property বুঝি
- [] Prototype-এ method রাখার কারণ বুঝি
- [] `new` keyword-এর core steps বুঝি
- [] `new` কীভাবে prototype connect করে বুঝি
- [] `new` কীভাবে `this` bind করে বুঝি
- [] Constructor return behavior বুঝি
- [] `new.target` বুঝি
- [] Factory vs Constructor difference বুঝি
- [] `user.__proto__ === User.prototype` relationship বুঝি
- [] `User.prototype.constructor` বুঝি
- [] Prototype chain আঁকতে পারি
- [] `Object.hasOwnProperty()` vs `in` basic difference বুঝি
- [] বুঝি কেন shared methods prototype-এ রাখা হয়